1. Design Approach

The main objective of this project was to design and simulate a three-phase voltage and overcurrent protection system using Proteus. I chose the Arduino Mega 2560 as the main controller because it provides enough ADC channels, digital I/O pins, I2C support, and serial communication interfaces for this project. The system monitors the voltage and current of the R, Y, and B phases and compares them with predefined thresholds.

I wanted the system to be interactive, so I included a 20×4 I2C LCD and a 4×4 keypad. The LCD provides a user interface that displays the voltage, current, and status of each phase. It also provides options for configuring the device settings. The keypad allows the user to adjust the voltage and current thresholds for each phase separately.

I also implemented serial communication so that the device can communicate with a computer through a serial port. The device can send relevant data, including voltage, current, phase status, and fault type, to the computer.

2. Implementation Process

I developed the circuit in Proteus and implemented the control logic in Arduino C++. The program reads the voltage and current sensor signals, calculates their RMS values, and checks for overvoltage, undervoltage, overcurrent, and sensor errors. I used separate fault states for the three phases so that a fault in one phase can trip its corresponding relay.

The grid AC voltage was stepped down using a transformer. The reduced AC voltage was further scaled using a voltage divider, and a DC offset was added to make the signal suitable for the MCU's ADC input. The final signals were connected to ADC pins A0, A1, and A2. A 550 Ω potentiometer was used to simulate overvoltage and undervoltage conditions.

The AC current was measured using ACS712 current sensors. To simulate an overcurrent condition, two lamps were connected in parallel across the sensor input. When both lamps were switched on simultaneously, the increased current could be used to test the overcurrent protection.

The I2C LCD and 4×4 matrix keypad act as the human-machine interface (HMI), allowing the user to monitor measurements and modify device settings. The LCD is connected through the I2C interface, while the keypad uses digital I/O pins.

Three motors are connected to the three-phase grid through protection relays. The MCU controls these relays through its digital I/O pins according to the detected fault conditions.

The user-defined thresholds are stored in EEPROM so that the settings can be retained. I also implemented serial communication to monitor measurements, device status, and fault events on a computer. This interface can also be used to record event logs for later analysis.

3. Results and Current Status

The simulation combines phase measurement, adjustable protection settings, LCD monitoring, and individual relay control. I also implemented serial communication for monitoring and debugging. Further testing is needed to verify all fault conditions and the complete serial communication path.

4. Conclusion

This project helped me understand the integration of measurement, fault detection, relay control, and serial communication in an embedded protection system. The next step is to verify the protection response and communication under different fault conditions.